透過 Google Antigravity 掌握 KCC 作業

來源：https://codelabs.developers.google.com/next26/kcc-ops-skill-antigravity?hl=zh-tw

步驟數：12

本程式碼研究室內容豐富，將帶您體驗 Antigravity，然後逐步詳細說明如何重建 KCC Ops Skill。

目錄

步驟 1 · 約 5 分鐘

1. 簡介

在本程式碼研究室中，您將瞭解 Google Antigravity (本文其餘部分稱為 Antigravity)，這是一個代理式開發平台，可將 IDE 帶往首重代理的紀元。

與只能自動完成程式碼行的標準程式設計助理不同，Antigravity 提供「任務控制中心」，可管理自主代理，規劃、編寫程式碼，甚至瀏覽網頁，協助您建構專案。

Antigravity 的設計以代理程式為優先，這項技術的前提是，AI 不只是編寫程式碼的工具，而是能夠自主規劃、執行、驗證及反覆處理複雜工程工作的角色，且幾乎不需要人為介入。

課程內容

- 安裝及設定 Antigravity。
- 瞭解 Antigravity 的重要概念，例如 Agent Manager、Editor 和 Browser 等。
- **從頭開始建構生產環境等級的 KCC Ops Skill**，安全且符合法規地管理 Google Cloud 資源。

軟硬體需求

目前 Antigravity 僅適用於個人 Gmail 帳戶的預先發布版。並提供免費配額，供您使用頂尖模型。

您必須在本機系統上安裝 Antigravity。這項產品適用於 Mac、Windows 和特定 Linux 發行版本。除了自己的機器，你還需要下列項目：

- Gmail 帳戶 (個人 Gmail 帳戶)。
- Google Cloud 帳戶和 Google Cloud 專案
- 支援 Google Cloud 控制台和 Cloud Shell 的網路瀏覽器，例如 Chrome

步驟 2 · 約 5 分鐘

2. 設定和需求

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

專案設定

建立 Google Cloud 專案

1. 在 Google Cloud 控制台的專案選取器頁面中，**選取或建立 Google Cloud 專案**。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何檢查專案是否已啟用計費功能。

本程式碼研究室適合各種程度的使用者和開發人員 (包括初學者) 參加。

步驟 3 · 約 10 分鐘

3. 安裝

如果您尚未安裝 Antigravity，請先安裝。目前這項產品處於預覽階段，您可以使用個人 Gmail 帳戶開始使用。

前往下載頁面，然後點選適用於您情況的作業系統版本。啟動應用程式安裝程式，並在電腦上安裝。安裝完成後，請啟動 Antigravity 應用程式。

設定期間的重要步驟：

- **選擇設定流程：**建議您在本程式碼研究室中從頭開始。
- **以審查為導向的開發 (建議)**：選擇這個選項。代理程式可以做出決定，並返回使用者要求核准，這對基礎架構作業至關重要。

接著，**設定編輯器**並**登入 Google**。最後，請接受《**使用條款**》。

步驟 4 · 約 20 分鐘

4. 基礎架構設定：GKE 和 Config Connector

如要建構技能，您需要 Google Cloud 環境，並手動安裝 **Config Connector (KCC)**，然後在**命名空間模式**中設定。讓您以 Kubernetes 物件的形式管理 GCP 資源。

步驟 0：準備環境

1. 叢集先決條件

建立新的 GKE 叢集，並啟用必要功能：

```
# Set your variables
export PROJECT_ID=$(gcloud config get-value project)
export CLUSTER_NAME="kcc-ops-cluster"
export REGION="us-central1"

# Create the cluster
gcloud container clusters create ${CLUSTER_NAME} \
    --region ${REGION} \
    --release-channel "regular" \
    --workload-pool=${PROJECT_ID}.svc.id.goog \
    --logging=SYSTEM \
    --monitoring=SYSTEM

# Get cluster credentials
gcloud container clusters get-credentials ${CLUSTER_NAME} --region ${REGION}
```

2. 安裝 Config Connector Operator

運算子會讓安裝項目保持最新狀態。

```
# Download the latest Config Connector operator
gcloud storage cp gs://configconnector-operator/latest/release-bundle.tar.gz release-
bundle.tar.gz

# Extract the bundle
tar zxvf release-bundle.tar.gz

# Install the operator (Standard Cluster)
kubectl apply -f operator-system/configconnector-operator.yaml
```

3. 設定命名空間模式

建立 `ConfigConnector` 資源來指定作業模式。

```
# configconnector.yaml
apiVersion: core.cnrm.cloud.google.com/v1beta1
kind: ConfigConnector
metadata:
  name: configconnector.core.cnrm.cloud.google.com
spec:
  mode: namespaced
  stateIntoSpec: Absent
```

```
kubectl apply -f configconnector.yaml
```

4. 建立身分和命名空間

在本實驗室中，我們將使用 `default` 命名空間和專屬的 Google 服務帳戶 (GSA)。

```
# Set your variables
export PROJECT_ID=$(gcloud config get-value project)
export NAMESPACE="default"

# Create the Google Service Account
gcloud iam service-accounts create kcc-identity --project ${PROJECT_ID}

# Grant permissions on the project
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
    --member="serviceAccount:kcc-identity@${PROJECT_ID}.iam.gserviceaccount.com" \
    --role="roles/owner"

# Grant Metric Writer permissions
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
    --member="serviceAccount:kcc-identity@${PROJECT_ID}.iam.gserviceaccount.com" \
    --role="roles/monitoring.metricWriter"

# Bind GSA to KSA via Workload Identity
gcloud iam service-accounts add-iam-policy-binding \
    kcc-identity@${PROJECT_ID}.iam.gserviceaccount.com \
    --member="serviceAccount:${PROJECT_ID}.svc.id.goog[cnrm-system/cnrm-controller-
manager-${NAMESPACE}]" \
    --role="roles/iam.workloadIdentityUser"
```

5. 設定命名空間

建立 `ConfigConnectorContext` 來監控命名空間。

```
# configconnectorcontext.yaml
apiVersion: core.cnrm.cloud.google.com/v1beta1
kind: ConfigConnectorContext
metadata:
  name: configconnectorcontext.core.cnrm.cloud.google.com
  namespace: default
spec:
  googleServiceAccount: "kcc-identity@${PROJECT_ID}.iam.gserviceaccount.com"
  stateIntoSpec: Absent
```

```
kubectl apply -f configconnectorcontext.yaml
```

6. 驗證安裝

等待控制器準備好 `default` 命名空間。

```
kubectl wait -n cnrm-system \
    --for=condition=Ready pod \
    -l cnrm.cloud.google.com/component=cnrm-controller-manager \
    -l cnrm.cloud.google.com/scoped-namespace=default
```

步驟 5 · 約 10 分鐘

5. 代理程式管理員：任務控管

Antigravity 是以開放原始碼的 Visual Studio Code (VS Code) 為基礎，但大幅改變使用者體驗，將重點從文字編輯轉移至代理程式管理。介面會分成兩個不同的主要視窗：`Editor` 和 `Agent Manager`。

服務專員管理員

啟動 Antigravity 後，使用者通常會看到 Agent Manager。這個介面是 **任務控管**資訊主頁。這項工具專為高階調度而設計，可讓開發人員在不同工作區或工作生成、監控及與多個非同步運作的代理互動。

在這個檢視畫面中，開發人員扮演架構師的角色。他們會定義高階目標。每項要求都會產生專屬的代理程式執行個體。使用者介面會以視覺化方式呈現這些平行工作流程，顯示每個代理的狀態、產生的**構件** (計畫、結果、差異) 和任何待核准的要求。

步驟 6 · 約 10 分鐘

6. Antigravity Browser 和 Artifacts

Antigravity 會在規劃及完成工作時建立**構件**。包括豐富的 Markdown 檔案、架構圖、圖片、瀏覽器錄影畫面和程式碼差異。

構件可解決「信任落差」

如果服務專員聲稱「我已修正錯誤」，開發人員先前必須閱讀程式碼才能驗證。在「反重力」中，代理程式會產生證明這點的構件。

構件

Antigravity 主要產生的構件如下：

- `Task Lists`：編寫程式碼前產生的結構化計畫。
- `Implementation Plan`：架構變更和技術細節。
- `Walkthrough`：變更摘要和測試方式。
- `Browser Recordings`：瀏覽器工作階段的影片記錄，用於驗證 UI。

Antigravity 瀏覽器

當代理程式需要與網路互動時，會叫用**瀏覽器子代理程式**。這個子代理程式可以點選、捲動、輸入及讀取控制台記錄。這項功能會使用專用模型，處理 Antigravity 管理的瀏覽器中開啟的網頁。

步驟 7 · 約 5 分鐘

7. 編輯器體驗

編輯器保留了 VS Code 的熟悉感，包括標準檔案總管、語法螢光標註和擴充功能生態系統。

編輯器主要功能：

- **自動完成**：按下 **Tab** 鍵即可接受智慧建議。
- **匯入分頁**：建議新增缺少的依附元件。
- **指令 (`Cmd + I`)**：使用自然語言觸發內嵌完成功能。
- **代理程式側邊面板 (`Cmd + L`)**：切換代理程式面板，使用 `@` 提問或參閱檔案。

步驟 8 · 約 10 分鐘

8. 提供意見

Antigravity 的核心功能是輕鬆收集意見回饋。您可以透過**Google 文件風格的註解**，向服務專員提供意見回饋。

在方案或工作新增註解時，請務必記得提交註解。這會引導代理程式朝您希望的方向發展。

步驟 9 · 約 30 分鐘

9. 建構 KCC 作業技能

瞭解平台後，接著來建構 **KCC Ops Skill**。

Kubernetes Config Connector (KCC) 可讓您將 GCP 資源視為 K8s 物件進行管理。不過，這需要安全防護措施，才能避免設定偏移、違規，以及意外重新建立資源。

步驟 1：建構技能

在工作區根目錄中，為技能建立目錄結構：

```
mkdir -p .agents/skills/kcc-ops/scripts
mkdir -p .agents/skills/kcc-ops/resources/policies/templates
mkdir -p .agents/skills/kcc-ops/resources/policies/constraints
```

步驟 2：撰寫 SKILL.md (大腦)

`SKILL.md` 定義代理程式的中繼資料和核心「黃金法則」。建立 `.agents/skills/kcc-ops/SKILL.md`：

```
---
name: kcc-ops
description: Assists with Config Connector (KCC) configuration, resource generation, and
troubleshooting on Google Cloud.
---

# Config Connector (KCC) Operations Skill

Use this skill to manage Google Cloud resources using Kubernetes-style configuration (Config
Connector).

## 🛑 GOLDEN RULE: Separate Generation from Application

**NEVER generate and apply a manifest in a single autonomous step.**

1. **Craft:** Write the generated manifest to a local file.
2. **Analyze:** Present the manifest to the user. Perform Impact Analysis and Dry Runs.
Explain the consequences of the change (e.g., "If this topic is deleted, the attached
subscription becomes orphaned").
3. **Wait:** Pause execution and explicitly wait for user permission to proceed.
4. **Apply:** Only run `kubectl apply` *after* the user has reviewed the manifest and the
impact analysis, and then unequivocally confirmed you should proceed.

## Core Responsibilities

0. **Context Verification**: Verify the execution context (cluster, namespace, GCP project,
user account) with the user before performing any operations.
1. **Installation & Health**: Verify KCC is properly installed and healthy on the target
cluster.
2. **Resource Inventory**: Query and summarize existing KCC resources within a namespace.
3. **Brownfield Bulk Export (Adoption)**: Export existing GCP project resources into valid
KCC YAML manifests.
4. **Manifest Generation**: Generate valid YAML manifests for GCP resources using KCC CRDs.
5. **Impact Analysis**: Identify ancillary services and resources (e.g., Cloud Run, Apps)
that depend on a resource being modified.
6. **Change Differentiation**: Generate diff summaries for resource edits to support change
control.
7. **Policy Compliance**: Vet KCC manifests against OPA/Gatekeeper policies.
8. **Troubleshooting**: Analyze resource status, and consult the troubleshooting guide to
resolve reconciliation issues.

## Guidelines for Operations

### 0. Context Verification

Before performing **any operations or executing commands** (including health checks), you
**MUST** verify the current execution context and obtain explicit user confirmation.

1. **Read Context:** Use commands like `kubectl config current-context`, `kubectl config view
--minify -o jsonpath='{.contexts[0].context.namespace}'`, `gcloud config get-value project`,
and `gcloud config get-value account` to determine the active environment.
2. **Present & Ask:** Show this information to the user clearly (e.g., "I see my context is
```

```
X, namespace is Y, project is Z, and account is A. Is this correct?").
3. **Wait:** Do not proceed with any other steps or scripts until the user has confirmed or provided corrections.

### 1. Installation & Health Check

Before performing operations, ensure the environment is ready:

- **Automation**: You MUST use `./scripts/check-health.sh` to verify namespaces, controllers, and CRDs. Do not use manual kubectl commands for health checks, as the script enforces standard formatting and context verification.

### 2. Resource Inventory & Discovery

To understand the current state of infrastructure:

- **Automation**: You MUST use `./scripts/inventory.sh` to get a summary table of all KCC resources. Do not use manual kubectl queries, as the script is optimized to securely discover all CRDs with context validation.

### 3. Manifest Structure

- All KCC resources belong to the `cnrm.cloud.google.com` API group.
- Use the `cnrm.cloud.google.com/project-id` annotation for cross-project resource management if not using Namespaced Mode.
- Always include `apiVersion`, `kind`, `metadata`, and `spec`.

### 4. Official Resource Reference (Agent Action)

When generating or troubleshooting manifests, you **must not guess** the API schema. Always consult the [Official Config Connector Reference](https://cloud.google.com/config-connector/docs/reference/overview) for the exact API version, kind, and required fields for the specific resource and cross reference with the official [github repository](https://github.com/GoogleCloudPlatform/k8s-config-connector/tree/master/config/crds/resources).

### 5. Troubleshooting Checklist

When a resource is not reconciling (check `kubectl get <kind> <name> -o yaml`):

- **Ready Condition**: Look for `status.conditions` where `type: Ready` and `status: "False"`.
- **Reason/Message**: Check the `reason` and `message` fields in the status conditions.
- **Consult the Guide**: Immediately check `./resources/troubleshooting-guide.md` for definitions of the error reason (e.g. `DependencyInvalid`, `ManagementConflict`) and follow its resolution steps.
- **Common Issues**:
  - Permissions: The KCC service account lacks IAM roles.
  - Quotas: GCP project quota exceeded.
  - Conflicts: Resource already exists or is managed by another tool.
  - Immutable Fields: Attempting to change a field that requires resource recreation. Look for "Update failed" errors related to immutable fields.
```

  - Reference Resolution: Check if the resource is waiting for a dependency (e.g., `referenced project not found`).

6. Impact Analysis (Ancillary Services)

Before modifying a resource (e.g., GCS Bucket, Pub/Sub Topic), verify whatElse depends on it:

- **Reference Search (Cluster-wide)**: Search for other KCC resources that reference the item.

  ```bash
  # Example: Find resources referencing a bucket named 'my-data-bucket'
  kubectl get-all -n <namespace> -o yaml | grep -C 5 "my-data-bucket"
  ```

- **IAM-based Analysis**: Check for IAM Service Accounts that have roles on the specific resource. A Cloud Run job or GKE Workload Identity might be using those permissions.
- **Common Ancillary Dependencies**:
  - **Storage Buckets**: Look for Cloud Run/GKE mounts (CSI), Cloud Functions triggers, or Dataflow jobs.
  - **Networks**: Check for Firewall rules, Forwarding rules, and GKE cluster assignments.
  - **IAM Policies**: Changing a policy might break access for external applications not managed by KCC.
- **Resource Graph**: Use `gcloud asset search-all-resources` to find resources that might have implicit links.

3. Policy Compliance & Best Practices

Evaluate KCC manifests against security and governance policies. The vetting tool supports three source modes:

- **Built-in Mode (Default)**: Uses the skill's high-fidelity `v1beta1` library (300+ Anthos constraints).
  - `Usage: ./scripts/vet-policy.sh <manifest-path>`
- **Remote Mode**: Clones and vets against an external Git repository.
  - `Usage: ./scripts/vet-policy.sh <manifest-path> <repo-url> [git-ref]`
  - ⚠️ **Note**: External libraries like the legacy GCP Policy Library may be out-of-date and cause schema validation errors with modern `gator`.
- **Local Mode**: Vets against a local directory of policies.
  - `Usage: ./scripts/vet-policy.sh <manifest-path> /path/to/local/policies`

Interaction Model:

1. Call `./scripts/vet-policy.sh` with the appropriate arguments.
2. Interpret the `=== KCC Best Practices ===` and `=== OPA/Gatekeeper ===` reports.
3. Supplement automated findings with manual review for specific security features not yet covered by OPA (e.g., `publicAccessPrevention: enforced`, `versioning: {enabled: true}`).

  ```bash
  # Run the skill's helper script (repo URL and branch are optional)
  ./scripts/vet-policy.sh manifest.yaml [policy-repo-url] [policy-ref]
  ```

Skill Assets

This skill includes additional resources to streamline operations:

- **`scripts/`**: Automation scripts (e.g., `vet-policy.sh`, `bulk-export.sh`).
- **`examples/`**: Reference KCC manifests (e.g., `restricted-bucket.yaml`).
- **`resources/`**: Common templates, documentation snippets, and troubleshooting guides (e.g., `troubleshooting-guide.md`).

4. Safety Rails for Applying Manifests

Before applying any KCC manifest update to an existing resource, you MUST:

- **Verify Immutable Fields**: Call `./scripts/verify-immutable.sh <manifest-path>` to detect updates to fields (like `location`, `name`, `project-id`) that trigger destructive resource recreation.
- **Explain Impact**: If destructive changes are detected, you MUST warn the user and explain the downtime/data loss implications before requesting approval.

5. Emergency Recovery & Troubleshooting

If a resource is stuck in a "Deletion" or "Error" state:

- **Check for Abondon Flag**: Check if the resource has the `cnrm.cloud.google.com/deletion-policy: abandon` annotation. If it does, you will need to remove the annotation and then force delete the resource.
- **Force Delete**: Call `./scripts/force-delete.sh <kind> <name> [namespace]` to bypass Kubernetes finalizers and remove the resource from the cluster.
- **Orphan Warning**: Inform the user that force-deleting a KCC object may leave an orphaned resource in Google Cloud that requires manual cleanup.

6. Change Differentiation

When editing an existing resource, always generate a diff to summarize the change for reviewers or Git history:

- **Local Diff**:

  ```bash
  # Diff a local file against the cluster state
  kubectl diff -f modified-resource.yaml
  ```

- **Commit Summary Template**:

  ```text
  [KCC Change] Update <ResourceName> (<Kind>)
  - Field 'spec.foo' changed from 'X' to 'Y'
  - Impact: Ancillary service <ServiceName> will see updated <Config>
  ```

9. Best Practices

- **Namespaced Mode**: Prefer namespaced mode for better isolation.
- **Sensitive Data**: Use `spec.credential.secretRef` or similar for sensitive fields.
- **Resource Naming**: Use consistent naming conventions that match your Kubernetes/GCP standards.
- **Annotations**:
  - `cnrm.cloud.google.com/deletion-policy: abandon`: Keep GCP resource on KCC deletion.
  - `cnrm.cloud.google.com/state-into-spec: absent`: Prevents KCC from syncing GCP state back into the Kubernetes object (useful for avoiding reconciliation loops on fields like node counts).

Common Resource Examples

Compute Instance

```yaml
apiVersion: compute.cnrm.cloud.google.com/v1beta1
kind: ComputeInstance
metadata:
  name: instance-sample
  annotations:
    cnrm.cloud.google.com/project-id: "my-project-id"
spec:
  machineType: n1-standard-1
  zone: us-central1-a
  bootDisk:
    initializeParams:
      sourceImage: projects/debian-cloud/global/images/family/debian-11
  networkInterface:
    - networkRef:
        name: default
```

Storage Bucket

```yaml
apiVersion: storage.cnrm.cloud.google.com/v1beta1
kind: StorageBucket
metadata:
  name: bucket-sample
spec:
  location: US
```

Pub/Sub Topic & Subscription

```yaml
apiVersion: pubsub.cnrm.cloud.google.com/v1beta1
kind: PubSubTopic
metadata:
  name: order-events-topic
```

```
---
apiVersion: pubsub.cnrm.cloud.google.com/v1beta1
kind: PubSubSubscription
metadata:
  name: order-processor-sub
spec:
  topicRef:
    name: order-events-topic
  ackDeadlineSeconds: 30
```

步驟 3：實作廣告空間工具

建立 `.agents/skills/kcc-ops/scripts/inventory.sh` 來探索 KCC 資源：

```
#!/bin/bash
# List all resources in the cnrm.cloud.google.com group
KCC_KINDS=$(kubectl api-resources --no-headers | awk '/\.cnrm\.cloud\.google\.com/ {print
$1}')
KCC_KINDS_CSV=$(echo "$KCC_KINDS" | paste -sd, -)

printf "%-40s %-30s %-10s %s\n" "KIND" "NAME" "READY" "STATUS/MESSAGE"
kubectl get "$KCC_KINDS_CSV" -A -o custom-
columns="KIND:.kind,NAME:.metadata.name,READY:.status.conditions[?
(@.type=='Ready')].status,MSG:.status.conditions[?(@.type=='Ready')].message" --ignore-not-
found --no-headers
```

步驟 4：新增政策審查邏輯

建立 `.agents/skills/kcc-ops/scripts/vet-policy.sh`。這個指令碼會使用 `gator` 根據 OPA 政策審查資訊清單：

```
#!/bin/bash
MANIFEST=$1
SKILL_ROOT=$(dirname "$(dirname "$0")")
POLICY_SRC="$SKILL_ROOT/resources/policies"

echo "=== OPA/Gatekeeper Policy Vetting ==="
if command -v gator >/dev/null 2>&1; then
    gator test -f "$MANIFEST" -f "$POLICY_SRC/templates" -f "$POLICY_SRC/constraints"
else
    echo "Gator not found. Skipping OPA audit."
fi
```

步驟 5：導入不可變更的欄位保護機制

這是重要的安全防護措施。建立 `.agents/skills/kcc-ops/scripts/verify-immutable.sh`：

```
#!/bin/bash
MANIFEST=$1
KIND=$(grep "^kind:" "$MANIFEST" | awk '{print $2}')
NAME=$(grep "name:" "$MANIFEST" | head -n 1 | awk '{print $2}')

# Check for changes in common immutable fields
IMMUTABLE_FIELDS=("location" "project-id" "name" "zone" "region")
TEMP_FILE=$(mktemp)
kubectl get "$KIND" "$NAME" -o yaml > "$TEMP_FILE" 2>/dev/null

for field in "${IMMUTABLE_FIELDS[@]}"; do
    NEW=$(grep "$field:" "$MANIFEST" | awk '{print $2}')
    OLD=$(grep "$field:" "$TEMP_FILE" | awk '{print $2}')
    if [ -n "$NEW" ] && [ -n "$OLD" ] && [ "$NEW" != "$OLD" ]; then
        echo "🧯 WARNING: Immutable field '$field' is changing! Potential resource
recreation."
    fi
done
rm "$TEMP_FILE"
```

步驟 6：緊急復原 (強制刪除)

建立 `.agents/skills/kcc-ops/scripts/force-delete.sh`：

```
#!/bin/bash
KIND=$1; NAME=$2; NS=${3:-default}
echo "Removing finalizers for $KIND/$NAME in $NS..."
kubectl patch "$KIND" "$NAME" -n "$NS" -p '{"metadata":{"finalizers":null}}' --type=merge
kubectl delete "$KIND" "$NAME" -n "$NS" --wait=false
```

步驟 7：完成資源

將所有指令碼設為可執行狀態：

```
chmod +x .agents/skills/kcc-ops/scripts/*.sh
```

步驟 10 · 約 10 分鐘

10. 測試新技能

現在，開始新對話並測試你的技能：

1. **探索**：`@kcc-ops Show me all KCC resources in my cluster.`
2. **法規遵循**：建立含有 StorageBucket 的 `bucket.yaml` 檔案。提問：`@kcc-ops Vet my bucket.yaml manifest.`
3. **安全性**：嘗試在 `bucket.yaml` 中更新現有 bucket 的 `location`。提問：`@kcc-ops Verify my bucket.yaml for immutable changes.`

請注意，AI 專員會智慧地選擇正確的腳本，並遵循 `SKILL.md` 中的「黃金法則」。

步驟 11 · 約 10 分鐘

11. 保護代理程式

讓 AI 代理存取終端機功能強大，但需要控制。

前往「Antigravity - Settings - Terminal」，然後瀏覽「Allow List」和「Deny List」。

- **允許清單**：在此新增 `ls`、`kubectl get` 和技能指令碼。
- **拒絕清單**：加入 `sudo`、`rm -rf` 或其他破壞性指令，確保代理「一律」會要求授權。

步驟 12 · 約 0 分鐘

12. 結論

恭喜！您已從安裝 Antigravity，進展到建構高精確度的 **KCC Operations Skill**。

您已學會：

- 如何使用自訂 Bash 工具擴充代理程式。
- 如何將作業「黃金法則」編碼為 `SKILL.md`。
- 如何為複雜的基礎架構管理作業提供安全防護。

後續步驟

使用更多 OPA 限制擴充 `resources/policies` 資料夾，或新增 `check-health.sh` 指令碼，自動檢查叢集是否已準備就緒！
